

ADIV-ContextSpace v0.1

A shared structure for human-AI situational awareness and objective resolution

Introduction

ADIV-ContextSpace is an approach for organizing objective-oriented work with shared situational awareness between human and AI, especially in software development.

Its main purpose is to support objective resolution with enough understanding to analyze, decide, implement, and validate the evolution of a solution. The contextual trail appears as a natural consequence of well-organized work.

The approach was born in the context of software development in coauthorship with AI agents. In this context, the work involves more than code: it also involves analysis, exploration, decision, documentation, prototyping, implementation, and validation, stages where AI can participate and collaborate as an agent.

The approach can also be applied to other domains where humans and agents need to build understanding, make decisions, and follow the evolution of a solution over time.

About the ADIV Acronym

In ADIV-ContextSpace, ADIV stands for:

- **Analyze**
- **Decide**
- **Implement**
- **Validate**

The ADIV cycle is the operational method used to move from an objective to a validated solution.

It expresses a natural and recurring sequence for solving problems and achieving objectives across different domains: understand the situation, choose a path, implement the choice, and check the result. The same pattern can be observed in decision-making and continuous-improvement models such as OODA and PDCA.

The ADIV acronym may appear in other contexts with different meanings, including Analysis, Design, Implementation, and Validation in software-development workflows. In ADIV-ContextSpace, the relevant distinction is that **D** means **Decision**: a recorded marker that links analysis, implementation, and validation inside a shared contextual space.

Core Idea

The main operational asset of ADIV-ContextSpace is **shared situational awareness**.

Situational awareness, in this context, is enough understanding about:

- Which objective is being pursued;
- Which context matters;
- Which references support the analysis;
- Which alternatives were explored;
- Which decision was made;
- What was implemented;
- How the implementation was checked;
- Which pending points still exist.

Context Space materializes this understanding in a practical structure of folders and files. This structure allows humans and agents to resume work from shared context.

It stores contextual artifacts that support solution development. Implementations are performed and maintained in their own origin repositories.

The ADIV Cycle

An ADIV cycle is a unit of work oriented toward an objective.

The cycle may start from any need that requires understanding, decision, and implementation: a bug, feature, research effort, legacy routine, business rule, document, integration, screen, report, conceptual review, or any other objective.

The cycle organizes work into four movements:

Analyze

Analyze means building enough understanding to decide.

At this stage, the focus is to understand the situation: existing sources, current behavior, evidence, rules, constraints, examples, documents, impacts, and alternatives.

Analysis may happen through conversation, source reading, file comparison, or exploration. When context needs to be preserved, it can be recorded in **adiv.md** or in auxiliary artifacts.

The **## Analyze** section should remain useful and readable. When analysis becomes too detailed, move dense parts to auxiliary **.md** files or focused artifacts, and reference them from **Analyze**.

Guiding question:

What do we need to understand in order to decide well?

Decide

Decide means choosing the actionable path.

In ADIV, the decision has a central role because it connects understanding to implementation. Before changing code, implementation files, or final artifacts, the decision must be recorded in the **## Decision** section of **adiv.md**.

This gate aligns human and AI on what will be implemented.

Guiding question:

What will we do?

Implement

Implement means executing the decision.

In software development, implementation usually happens in the system's own repository. ADIV records what was done according to the decision in the **## Implementation** section of **adiv.md**, preserving the situational awareness of the implementation.

One ADIV may contain several rounds of decision, implementation, and validation to resolve the objective, at the human's discretion.

Guiding question:

How was the decision materialized?

Validate

Validate means checking whether the implementation meets the objective and the recorded decision.

Validation is the point in the cycle where checks performed by AI are recorded in the **## Validation** section of **adiv.md**. It is up to the human to evaluate, according to their own criteria, whether the objective was achieved or whether the work should continue. Their validations may also be recorded, at their discretion.

Guiding question:

Does the result meet the objective, or should the cycle continue?

Contextual Structure

The default Context Space structure organizes work by contextual location and actionable cycle:

```
adiv_contextspace/  
  project/  
    division/  
      grouping/  
        cycle/  
          adiv.md  
          reference/  
            reference.md  
          exploration/  
            exploration.md  
          candidate/  
            candidate.md  
          official/  
            official.md  
          record/  
            record_cN.md # only when an item is marked [Cn+]
```

The levels above the cycle locate the context:

- **project**: product, system, book, initiative, client, or solution.
- **division**: module, domain, part, area, or process.
- **grouping**: routine, screen, flow, rule, chapter, theme, integration, or another local cut.
- **cycle**: actionable objective with `adiv.md`.

Everything that has **`adiv.md`** is an ADIV cycle. Everything above the cycle is location context.

The objective may involve analysis, exploration, prototyping, documentation, coding, and other solution-development stages, in one ADIV or in multiple ADIVs related by reference.

The `adiv.md` File

`adiv.md` is the operational entry point of the cycle.

It should preserve enough synthesis to allow continuity of work. When there are long specifications, extensive models, logs, payloads, scripts, comparisons, or full proposals, those materials should stay in the corresponding auxiliary folders.

As the cycle's living operational context, it records the operational situation of that cycle: what is known, what was decided, what was done, what was validated, and what is still pending.

Suggested structure:

```
# <Cycle title>

## Identification
- Created on:
- Participants:
- Origin:

## Objective

## Context

## Analysis

## Decision

## Implementation

## Validation

## Pending
```

Internal Tags

Internal tags are used inside `adiv.md` to preserve traceability between decisions, implementations, and validations.

Tags `[Cn]` and `[Cn+]` connect a decision to the implementation and validation entries that belong to the same internal round of the ADIV cycle.

```
[Cn]  = item belongs to internal decision/round Cn.
[Cn+] = item belongs to Cn and has expanded contextual record in record/
record_cn.md.
```

The tag starts in `## Decision`.

The `## Implementation` and `## Validation` sections reuse the same tag when they implement or check that decision.

Example:

```
## Decision
- Use `BaseInvoiceIntegrationService.cs` as the reference implementation for
`NewInvoiceIntegrationService.cs`. [C1]

## Implementation
- Created `NewInvoiceIntegrationService.cs` from the reference implementation. [C1]
```

```
## Validation
```

```
- Checked file creation and preservation of the expected structure. [C1]
```

Tags are preferably generated and maintained by AI because they are part of the cycle's operational traceability.

Expanded Record

record/ exists only to keep **adiv.md** compact when a decision, implementation, or validation needs expanded operational detail.

An item marked with **[Cn+]** indicates that there is an expanded contextual record file for that round:

```
record/record_c5.md
```

This file may detail decision, implementation, and validation. Consolidated specifications belong in **official/**. Evidence, comparisons, and hypotheses belong in **reference/** or **exploration/**, according to the state of the material.

Auxiliary Artifacts

Auxiliary folders exist to support the objective. They organize artifacts by state, role, and maturity:

- **reference/**: materials used to build understanding.
- **exploration/**: drafts, hypotheses, alternatives, studies, and intermediate models.
- **candidate/**: proposal mature enough for evaluation.
- **official/**: understanding or artifact treated as valid by the human in the context of the cycle.
- **record/**: expanded contextual record on demand for decisions, implementations, and validations marked with **[Cn+]**.

Each auxiliary folder may include a general **.md** file with the same name as the folder, such as **reference/reference.md**, **exploration/exploration.md**, **candidate/candidate.md**, or **official/official.md**. These files can be used as general entry points for the folder, supporting or complementing **adiv.md** without duplicating it.

These folders are not a pipeline or mandatory sequence. An artifact can be created directly in the folder that best matches its role and maturity.

These folders are available states, used on demand, at the human's discretion.

official is a maturity state defined at the human's discretion.

Chaining Between Cycles

ADIV cycles are independent, but they may reference each other.

One cycle may use links or copies of official artifacts, evidence, or decisions from another cycle as reference when they are used as input. This allows knowledge to evolve without unnecessary duplication.

Role of the Human and AI

The human defines the objective, the maturity criterion, the sufficiency of the solution, and the final validation.

AI supports analysis, organizes context, records decisions, implements authorized changes, maintains internal tags, performs checks, and preserves operational continuity.

When AI is in direct interaction with the human, it must record the decision before implementation and wait for direct implementation instruction. This alignment is part of the method itself.

Short Example

A cycle to fix an incorrect discount calculation in a sales order routine could have:

```
erp_system/  
  sales/  
    sales_order/  
      fix_order_discount_calculation/  
        adiv.md  
        reference/  
          reference.md  
          evidence_sales_screen_order_11042.png  
          evidence_printed_order_11042.pdf  
        exploration/  
          exploration.md  
        candidate/  
          candidate.md  
        official/  
          official.md  
        record/
```

The operational synthesis in **adiv.md** could be:

```
# Fix Order Discount Calculation  
  
## Identification  
- Created on: 2026-06-23  
- Participants: Adao, Codex
```

- Origin: Ticket `SUP-18427`

Objective

Fix an incorrect discount calculation in the sales order routine.

Context

A user reported a difference in the calculated discount after selecting the customer in the sales order routine. For order `11042`, the sales order screen and the printed order show different totals after the discount is applied.

Analysis

- The issue was observed in order `11042`.
- Use `reference/evidence\_sales\_screen\_order\_11042.png` and `reference/evidence\_printed\_order\_11042.pdf` as evidence for the calculation difference.
- The discount shown on the sales screen does not match the printed order total.
- The discount should be calculated according to the rule of the selected payment method.

Decision

- Adjust `C:\erp\_system\src\sales\SalesOrderTotals.cs` so the sales screen and printed order use the same discount base and rounding rule. [C1]
- Review `C:\erp\_system\src\sales\SalesOrderPrint.cs` to ensure it consumes the calculated discount total instead of recalculating it independently. [C1]

Implementation

- Changed `C:\erp\_system\src\sales\SalesOrderTotals.cs` to centralize the discount base and rounding rule used by the sales order total. [C1]
- Changed `C:\erp\_system\src\sales\SalesOrderPrint.cs` to use the discount total calculated by the sales order routine. [C1]

Validation

- Reviewed the implementation against the recorded decision and compiled the system repository without errors. [C1]
- The human tested the sales order flow and confirmed that the objective was achieved. [C1]

A Broader Conceptual Example

The next example shows only part of a broader ADIV cycle. In this kind of cycle, the work may involve not only code implementation, but also artifacts at different maturity levels, including exploratory notes, candidate proposals, official artifacts, and expanded decision records.

A cycle to implement a new salesperson registry could have:

```
erp_system/  
  sales/  
    salesperson_registry/  
      implement_salesperson_registry/  
        adiv.md  
        reference/  
          reference.md
```

```

exploration/
  exploration.md
  mockup_salesperson_registry_v1.png
candidate/
  candidate.md
  salesperson_registry_proposal.md
official/
  official.md
  salesperson_registry_requirements.md
  salesperson_registry_reference_screen.png
  salesperson_registry_schema.sql
record/
  record_c1.md

```

In **adiv.md**, one part of the cycle could be:

```

...

## Decision
- Consolidate the salesperson registry requirements in `official/
salesperson_registry_requirements.md`, based on the human-AI analysis and the
mockups explored in `exploration/`. [C1+]
- Implement the salesperson registry using the standard MVC structure in the Sales
module. [C2]
- Create the list and edit views following `official/
salesperson_registry_reference_screen.png`. [C3]

## Implementation
- Created `official/salesperson_registry_requirements.md` with the requirements
consolidated through human-AI collaboration. [C1+]
- Created `C:\erp_system\src\Sales\Models\Salesperson.cs` with the salesperson
entity and validation rules. [C2]
- Created `C:\erp_system\src\Sales\Controllers\SalespersonController.cs` with list,
create, edit, save, and delete actions. [C2]
- Created `C:\erp_system\src\Sales\Views\Salesperson\Index.cshtml` for the list/
search view. [C3]
- Created `C:\erp_system\src\Sales\Views\Salesperson\Edit.cshtml` for the create/
edit form. [C3]

...

```

These examples show the boundary: ADIV preserves context, decisions, implementation records, validation, supporting artifacts, and optional expanded records for decisions that need more detail; the changed code remains in the system repository.

Principles

- Shared situational awareness is the main operational asset.
- Understanding precedes implementation.
- Explicit decision guides implementation.

- Context Space supports objective resolution.
 - The contextual trail is a consequence of the work.
 - Auxiliary artifacts exist on demand.
 - Artifact states are used on demand.
 - Implementations live in their origin repositories.
 - The human defines maturity, sufficiency, and final validation.
 - ADIV should increase continuity with enough context.
-

Summary

ADIV-ContextSpace organizes context to resolve objectives with shared situational awareness.

ADIV is the cycle that conducts work through analysis, decision, implementation, and validation.

adiv.md is the living operational context of the cycle.

Auxiliary folders store contextual artifacts by state and maturity, used when they help the work.

Code and other implementations remain in their origin repositories.

Adao Aparecido Ernesto
June/2026

X: <https://x.com/adaoaper>

This document is part of ADIV-ContextSpace.

https://github.com/adaoaper/adiv_contextspace